

Reviewer Pack — Minimal Local Chromatic Number Correlation with Resolution P...

Entry 7dbdb07b5da5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 03:54:44 UTC

Minimal Local Chromatic Number Correlation with Resolution Proof Width

Entry ID: 7dbdb07b5da5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 03:54:44 UTC

1. Conjecture

Field A (mathematical branch): Graph Theory (Local Chromatic Number) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For any given Boolean satisfiability instance, the local chromatic number of its induced graph is linearly correlated with its resolution proof width. Specifically, for instances with n variables and m clauses, the local chromatic number $L(G)$ satisfies $|L(G) - w(\varphi)| \leq k$, where $w(\varphi)$ is the resolution proof width of the associated Tseitin formula φ_G and k is a constant.

Rationale (proposer's reasoning):

The local chromatic number captures the minimum number of colors needed to color the vertices of a graph such that no two adjacent vertices share the same color. This invariant may expose structural properties in the graph related to its ability to be satisfied, potentially leading to insights into resolution proof complexity. A linear correlation between these quantities could suggest a fundamental connection between combinatorial properties of graphs and the complexity of their satisfiability.

Taxonomy category: graph_theory_resproof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9c7614e94a7cc5b0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the given Boolean satisfiability instance, if the absolute difference between the local chromatic number $L(G)$ of its induced graph and the resolution proof width $w(\varphi_G)$ is less than or equal to a constant k across at least 80% of 30 randomly generated instances, it supports the conjecture. If any seed produces an absolute difference greater than k for the metric $|L(G) - w(\varphi_G)|$, it falsifies the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'local chromatic number' AND 'resolution proof width' AND graph theory' - 'Boolean satisfiability' AND 'induced graph local chromatic number' AND 'proof complexity' - 'Tseitin formula' AND 'local chromatic number' correlation WITHIN 1 sentence 'resolution proof width'

Top relevant hits considered: - [<http://arxiv.org/abs/2009.01996v1>] Approaches Which Output Infinitely Many Graphs With Small Local Antimagic Chromatic Number - [<http://arxiv.org/abs/1805.12181v1>] Computing Small Unit-Distance Graphs with Chromatic Number 5 - [<http://arxiv.org/abs/2504.01295v5>] A Spectral Lower Bound on Chromatic Numbers using p -Energy - [<http://arxiv.org/abs/1103.5740v2>] Generating and Searching Families of FFT Algorithms - [<http://arxiv.org/abs/cs/0701007v1>] On the Complexity of the Circular Chromatic Number - [<http://arxiv.org/abs/2311.08194v1>] On the Quantum Chromatic Numbers of Small Graphs - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(variables, clauses):
        tseitin_vars = {}
        tseitin_clauses = []

        for var in variables:
            tseitin_var = f"t{var}"
            tseitin_vars[var] = tseitin_var
            tseitin_clauses.append((tseitin_var,))

        for clause in clauses:
            tseitin_clause = []
            for literal in clause:
                if literal.startswith('not '):
```

```

        var = literal[4:]
        tseitin_clause.append(f"not {tseitin_vars[var]}")
    else:
        tseitin_clause.append(tseitin_vars[literal])
    tseitin_clauses.append(tuple(tseitin_clause))

tseitin_formula_str = " and ".join(" or ".join(clause) for clause in tseitin_clauses)
return tseitin_vars, tseitin_formula_str

def resolution_prove(formula):
    clauses = formula.split(" and ")
    while True:
        new_clause = None
        for i in range(len(clauses)):
            for j in range(i + 1, len(clauses)):
                clause_i = set(clauses[i].split(" or "))
                clause_j = set(clauses[j].split(" or "))
                if any(not (not p) in clause_j for p in clause_i):
                    new_clause = " or ".join(p for p in clause_j if p not in clause_i)
                    break
            if new_clause:
                break
        if new_clause is None:
            return len(clauses)
        clauses.append(new_clause)

def local_chromatic_number(graph):
    n = len(graph)
    colors = [-1] * n

    def dfs(node, color):
        stack = [node]
        while stack:
            node = stack.pop()
            if colors[node] == -1:
                colors[node] = color
                for neighbor in graph[node]:
                    if colors[neighbor] == -1:
                        stack.append(neighbor)

    for i in range(n):
        if colors[i] == -1:
            dfs(i, 0)

    return max(colors) + 1

def generate_instance(n, m):
    variables = set()
    clauses = []
    for _ in range(m):
        clause = random.sample(variables | {f"not {var}" for var in variables}, random.randint(1, n))
        clauses.append(clause)
    return variables, clauses

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0

```

```

instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    m = random.randint(n, 2 * n)
    variables, clauses = generate_instance(n, m)
    tseitin_vars, tseitin_formula_str = tseitin_formula(variables, clauses)
    graph = {i: set() for i in range(len(variables))}
    for clause in clauses:
        for literal in clause:
            if literal.startswith('not '):
                var = literal[4:]
                graph[tseitin_vars[var]].add(tseitin_vars[literal])
            else:
                graph[tseitin_vars[literal]].add(tseitin_vars[var])

    local_chromatic = local_chromatic_number(graph)
    resolution_width = resolution_prove(tseitin_formula_str)
    metric_value = abs(local_chromatic - resolution_width)

    total_metric_value += metric_value
    instances_tested += 1
    n_max = max(n_max, n)

    if metric_value > k:
        conjecture_holds = False
        counterexample = f"n={n}, m={m}, local_chromatic={local_chromatic}, resolution_width={resolution_width}"

return {
    "metric_name": "Absolute difference between local chromatic number and resolution proof width",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):

```

```

        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1452b051.py", line 135, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1452b051.py", line 97, in run_trial
    variables, clauses = generate_instance(n, m)
                          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1452b051.py", line 84, in generate_instance
    clause = random.sample(variables | {f"not {var}" for var in variables}, random.randint(1, n))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. "
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12540
2	preregistration	ollama_remote	glm4:latest	0	0	9746
3	novelty	ollama_remote	glm4:latest	0	0	8775
4	novelty	ollama_remote	glm4:latest	0	0	13792
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22356
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15098
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13263

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14646
9	judge	ollama_remote	glm4:latest	0	0	11902

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122119 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7dbdb07b5da5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7dbdb07b5da5.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7dbdb07b5da5.tar.gz (if generated)